

Les moniteurs (Hoare, 1974)

Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

Vous avez dit «moniteur»?

- Mécanisme riche de synchronisation
 - ▶ Mécanismes non structurés : «sémaphores» de Dijkstra (1965)
 - ▶ Facile à mettre en œuvre au niveau système
 - ▶ Plus difficile à utiliser
- Mécanisme «structurable» de synchronisation
 - ▶ Un modèle adaptable à différentes situations

«Presque» la notion sous-jacente au «synchronized»

- Introduit en 1974 par T. Hoare (et P. Hansen?)
 - ▶ Expérimentation autour du Concurrent Pascal (1975)

Plus riche que le seul «synchronized»

- S'appuie sur cette notion
- Implémentation aisée en Java

Définition d'un moniteur

3

Ensemble de «variables d'état»

-  Définissent le contexte du moniteur

Procédures manipulant ces variables

-  Un constructeur initialisant le contexte
-  Procédures externes

Conditions

-  Pour bloquer/réveiller les «utilisateurs» du moniteur
-  File d'attente de processus/threads
-  Trois opérations

- ▶ **C.attendre** → mise en attente sur C
- ▶ **C.vide** → rend TRUE si personne n'attend sur C
- ▶ **C.signaler** → réveille le plus ancien processus en attente de C (si $\neg \emptyset$)

Définition d'un moniteur

3

Ensemble de «variables d'état»

- Définissent le contenu

Procédures m

- Un constructeur in
- Procédures extern

Conditions

- Pour bloquer/réveiller les «utilisateurs» du moniteur
- File d'attente de processus/threads
- Trois opérations

- ▶ **C.attendre** → mise en attente sur C
- ▶ **C.vide** → rend TRUE si personne n'attend sur C
- ▶ **C.signaler** → réveille le plus ancien processus en attente de C (si $\neg \emptyset$)



Attention au principe primordial

Accès à une procédure en **exclusion mutuelle**

≠

Un seul **processus actif** dans le moniteur

Illustration (simple)

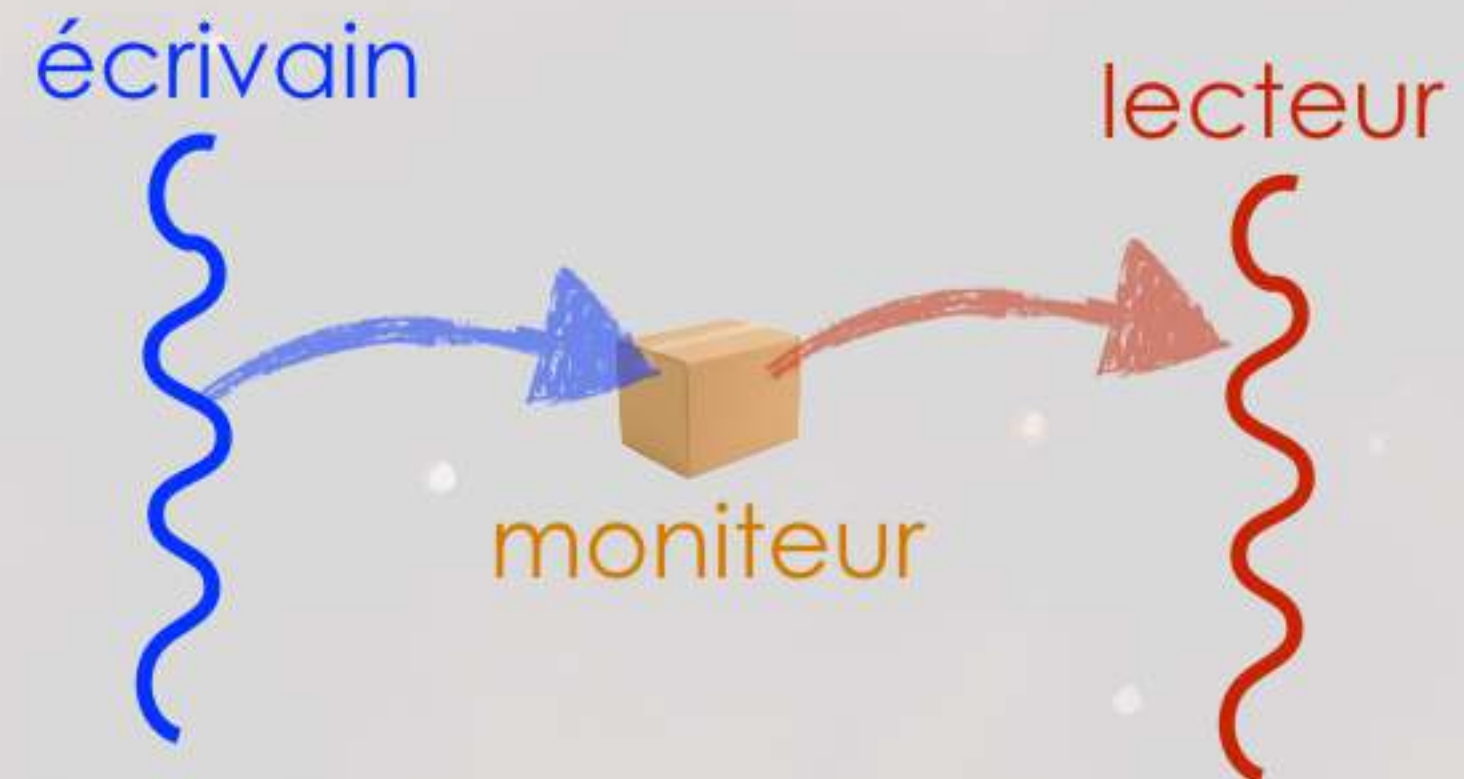
4

Un processus écrivain et un processus lecteur

 L'écrivain signale au lecteur qu'il peut lire une donnée

 Questions

- ▶ Quel contexte d'exécution pour le moniteur?
- ▶ Quelles primitives de manipulation?



Le moniteur (sync)

 Contexte

- ▶ termine (booléen)

 Primitives

- ▶ finEcrire() et debutLire()

 Condition

- ▶ fini

Mise en œuvre de l'exemple

5

Le moniteur (sync)

```
constructeur init() {  
    termine = false;  
}  
  
void finEcrire() {  
    termine = true;  
    fini.signaler();  
}  
  
void debutLire () {  
    if (! termine) {  
        fini.attendre()  
    }  
}
```

L'écrivain

```
...  
variablePartagee = calcul();  
sync.finEcrire();  
...
```

Le lecteur

```
...  
sync.debutLire();  
maVariable = variablePartagee;  
...
```


Première exécution possible

6

L'écrivain

Le lecteur

```
...  
variablePartagee = calcul();  
termine = true;  
fini.signaler();  
...
```

variablePartagee

```
...  
if (! termine) {  
    fini.attendre()  
}  
maVariable = variablePartagee;  
...
```

On considère

Le moniteur a été initialisé
L'écrivain arrive le premier

Second

On considère

Le moniteur a été initialisé
Le lecteur arrive le premier

7

L'écrivain

Le lecteur



En guise de conclusion...

8

Le moniteur est un objet adaptable

-  Il faut réfléchir aux synchronisations à résoudre



Notion cruciale

-  Un seul «fil d'exécution» actif dans les procédures du moniteur
 - Plus fin que de l'exclusion mutuelle

Contexte

-  Aussi riche que possible

Conditions

-  Autant que l'on souhaite
 - Bonne question = «combien de files d'attente souhaitons-nous?»